BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 10

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement: Design and implement deep learning application for audio classification that incorporates Speech to Text and Text to Speech using appropriate deep learning architecture and evaluate its performance using suitable metrics.

Algorithm:

1. Start the program and import the required libraries.
2. Load the RAVDESS speech emotion dataset and assign emotion labels to audio files.
3. Extract MFCC (Mel-Frequency Cepstral Coefficients) features from each audio file.
4. Encode emotion labels and split the dataset into training and testing sets.
5. Build and compile a CNN model for audio emotion classification.
6. Train the CNN model using the training dataset.
7. Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
8. Select an audio sample and predict its emotion using the trained model.
9. Convert the audio speech into text using the Whisper model.
10. Convert the recognized text and predicted emotion into speech using gTTS.
11. Display the predicted emotion, recognized text, and evaluation results, and save the trained model.
12. Stop the program.

Code :

```
# Install Required Libraries

%pip install -q numpy librosa scikit-learn tensorflow matplotlib
seaborn transformers torch gTTS


# Import Libraries and Initialize Random Seeds

import os
import glob
import numpy as np
import librosa
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    f1_score
)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, Flatten, Dense, Dropout
)
from tensorflow.keras.callbacks import EarlyStopping

np.random.seed(42)
tf.random.set_seed(42)


# Load Audio Files from the RAVDESS Dataset

DATASET_PATH = r"D:\Audio_Speech_Actors_01-24"

audio_files = glob.glob(
    os.path.join(DATASET_PATH, "**", "*.wav"),
```

```
    recursive=True
)

print("Total audio files:", len(audio_files))

if not audio_files:
    raise FileNotFoundError(
        "No WAV files found. Check DATASET_PATH and extract the ZIP
file."
    )


# Extract Audio File Paths and Corresponding Emotion Labels

emotion_map = {
    "01": "neutral",
    "02": "calm",
    "03": "happy",
    "04": "sad",
    "05": "angry",
    "06": "fearful",
    "07": "disgust",
    "08": "surprised"
}

files = []
labels = []

for file in audio_files:
    parts = os.path.basename(file).split("-")

    if (
        len(parts) == 7
        and parts[0] == "03"
        and parts[1] == "01"
        and parts[2] in emotion_map
    ):
        files.append(file)
        labels.append(emotion_map[parts[2]])

print("Speech recordings:", len(files))

if not files:
    raise ValueError(
```

```
        "No matching speech recordings found. Check the dataset
folder."
    )


# Extract MFCC Features for Audio Classification

def extract_mfcc(file):
    audio, sr = librosa.load(file, sr=22050)

    mfcc = librosa.feature.mfcc(
        y=audio,
        sr=sr,
        n_mfcc=40
    )

    if mfcc.shape[1] < 174:
        mfcc = np.pad(
            mfcc,
            ((0, 0), (0, 174 - mfcc.shape[1]))
        )
    else:
        mfcc = mfcc[:, :174]

    return mfcc


X = np.array([extract_mfcc(file) for file in files])

encoder = LabelEncoder()
y = encoder.fit_transform(labels)

X = X[..., np.newaxis]

print("Feature shape:", X.shape)
print("Emotion classes:", encoder.classes_)


# Split the Dataset into Training and Testing Sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))


# Design and Compile the CNN Model for Emotion Classification

model = Sequential([
    tf.keras.layers.Input(shape=(40, 174, 1)),

    Conv2D(32, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(64, activation="relu"),
    Dropout(0.3),

    Dense(len(encoder.classes_), activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()


# Train the CNN Model Using Audio Features

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=32,
    callbacks=[
        EarlyStopping(
            monitor="val_loss",
```

```
            patience=5,
            restore_best_weights=True
        )
    ]
)


# Plot Training and Validation Accuracy and Loss

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.title("Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training")
plt.plot(history.history["val_loss"], label="Validation")
plt.title("Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.tight_layout()
plt.show()


# Evaluate the Model Using Accuracy, F1-Score, and Confusion Matrix

y_pred = np.argmax(
    model.predict(X_test, verbose=0),
    axis=1
)

print("Test Accuracy:", accuracy_score(y_test, y_pred))
print("Macro F1-Score:", f1_score(
    y_test, y_pred, average="macro"
))

print("\nClassification Report:")
print(classification_report(
```

```
    y_test,
    y_pred,
    labels=np.arange(len(encoder.classes_)),
    target_names=encoder.classes_,
    zero_division=0
))

cm = confusion_matrix(
    y_test,
    y_pred,
    labels=np.arange(len(encoder.classes_))
)

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=encoder.classes_,
    yticklabels=encoder.classes_
)

plt.xlabel("Predicted Emotion")
plt.ylabel("Actual Emotion")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()


# Predict the Emotion and Confidence Score of an Audio Sample

# Select a sample from the test dataset
sample_index = 0
sample_path = files[
    train_test_split(
        np.arange(len(files)),
        test_size=0.2,
        random_state=42,
        stratify=y
    )[1][sample_index]
]

mfcc = extract_mfcc(sample_path)
mfcc = mfcc[np.newaxis, ..., np.newaxis]
```

```
prediction = model.predict(mfcc, verbose=0)[0]

emotion = encoder.inverse_transform([
    np.argmax(prediction)
])[0]

print("Audio File:", sample_path)
print("Predicted Emotion:", emotion)
print("Confidence:", round(float(np.max(prediction)) * 100, 2), "%")


# Perform Speech-to-Text Conversion Using Whisper

from transformers import pipeline

transcriber = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-tiny.en"
)

# Load audio using librosa to avoid the FFmpeg file-reading error
audio, sr = librosa.load(sample_path, sr=16000)

# Pass audio samples directly to Whisper
result = transcriber({
    "raw": audio.astype(np.float32),
    "sampling_rate": sr
})

text = result["text"]

print("Recognized Speech:", text)


# Perform Text-to-Speech Conversion Using gTTS

from gtts import gTTS
from IPython.display import Audio, display

result_text = (
    f"The recognized speech is {text}. "
    f"The predicted emotion is {emotion}."
)

tts = gTTS(result_text, lang="en")
```

```
tts.save("output.mp3")

print("Generated Speech:")
display(Audio("output.mp3", autoplay=False))


#Save the Trained CNN Model for Future Use

model.save("audio_emotion_model.keras")
print("Model saved successfully.")
```

Output :

- **Load and Extract dataset:**

```
Total audio files: 1440
```

[32]:
```
# Extract Audio File Paths and Corresponding Emotion Labels
emotion_map = {
    "01": "neutral", "02": "calm",
    "03": "happy", "04": "sad",
    "05": "angry", "06": "fearful",
    "07": "disgust", "08": "surprised"
}

files, labels = [], []

for file in audio_files:
    parts = os.path.basename(file).split("-")

    if len(parts) == 7 and parts[0] == "03" and parts[1] == "01":
        files.append(file)
        labels.append(emotion_map[parts[2]])

print("Speech recordings:", len(files))
```

```
Speech recordings: 1440
```

[34]:
```
# Extract MFCC Features for Audio Classification
def extract_mfcc(file):
    audio, sr = librosa.load(file, sr=22050)
    mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=40)

    if mfcc.shape[1] < 174:
        mfcc = np.pad(mfcc, ((0, 0), (0, 174 - mfcc.shape[1])))
    else:
        mfcc = mfcc[:, :174]

    return mfcc

X = np.array([extract_mfcc(f) for f in files])
encoder = LabelEncoder()
y = encoder.fit_transform(labels)

X = X[..., np.newaxis]

print("Feature shape:", X.shape)
print("Number of emotions:", len(encoder.classes_))
```

```
Feature shape: (1440, 40, 174, 1)
Number of emotions: 8
```

- **Distinct classes**

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 40, 174, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 20, 87, 32)	0
conv2d_3 (Conv2D)	(None, 20, 87, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 10, 43, 64)	0
flatten_1 (Flatten)	(None, 27520)	0
dense_2 (Dense)	(None, 64)	1,761,344
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 8)	520

Total params: 1,780,680 (6.79 MB)

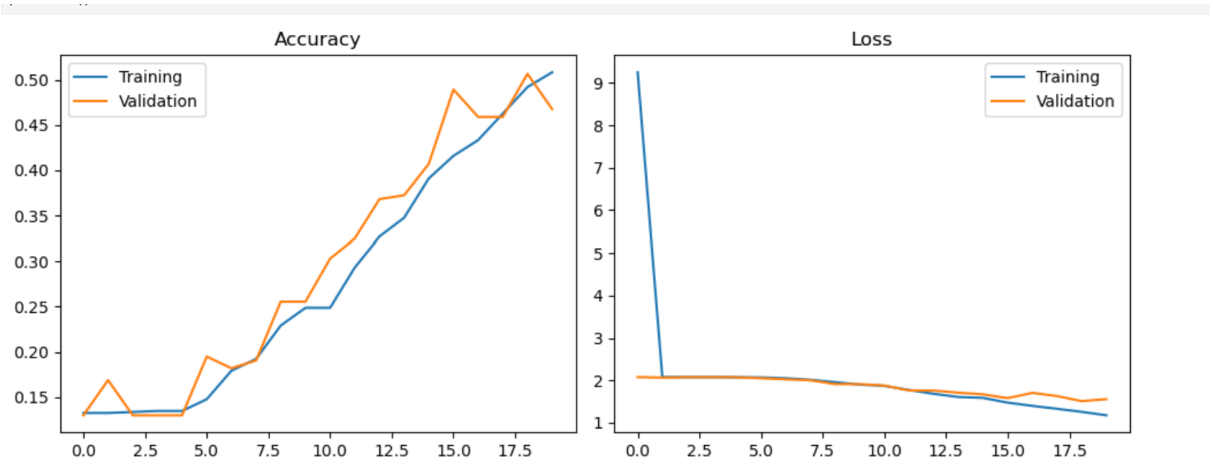
Trainable params: 1,780,680 (6.79 MB)

Non-trainable params: 0 (0.00 B)

- **Training Model**

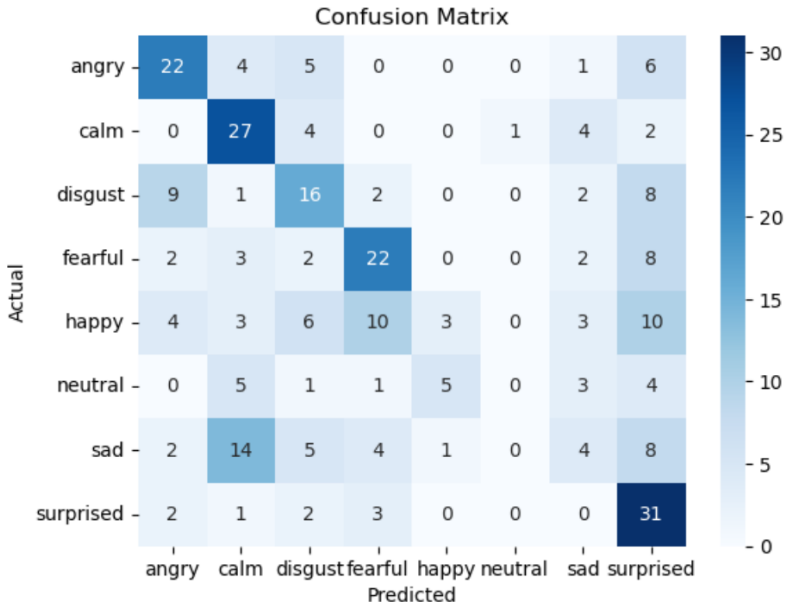
```
Epoch 1/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 5s 80ms/step - accuracy: 0.1325 - loss: 9.2526 - val_accuracy: 0.1299 - val_loss: 2.0784
Epoch 2/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 73ms/step - accuracy: 0.1325 - loss: 2.0786 - val_accuracy: 0.1688 - val_loss: 2.0661
Epoch 3/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.1336 - loss: 2.0772 - val_accuracy: 0.1299 - val_loss: 2.0761
Epoch 4/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 67ms/step - accuracy: 0.1346 - loss: 2.0762 - val_accuracy: 0.1299 - val_loss: 2.0749
Epoch 5/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 68ms/step - accuracy: 0.1346 - loss: 2.0745 - val_accuracy: 0.1299 - val_loss: 2.0738
Epoch 6/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 69ms/step - accuracy: 0.1477 - loss: 2.0701 - val_accuracy: 0.1948 - val_loss: 2.0517
Epoch 7/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 65ms/step - accuracy: 0.1792 - loss: 2.0522 - val_accuracy: 0.1818 - val_loss: 2.0265
Epoch 8/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 64ms/step - accuracy: 0.1922 - loss: 2.0147 - val_accuracy: 0.1905 - val_loss: 2.0038
Epoch 9/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 3s 68ms/step - accuracy: 0.2291 - loss: 1.9592 - val_accuracy: 0.2554 - val_loss: 1.9147
Epoch 10/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 65ms/step - accuracy: 0.2486 - loss: 1.9019 - val_accuracy: 0.2554 - val_loss: 1.9128
Epoch 11/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 67ms/step - accuracy: 0.2486 - loss: 1.8714 - val_accuracy: 0.3030 - val_loss: 1.8789
Epoch 12/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 64ms/step - accuracy: 0.2932 - loss: 1.7738 - val_accuracy: 0.3247 - val_loss: 1.7629
Epoch 13/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 64ms/step - accuracy: 0.3268 - loss: 1.6838 - val_accuracy: 0.3680 - val_loss: 1.7596
Epoch 14/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 64ms/step - accuracy: 0.3474 - loss: 1.6092 - val_accuracy: 0.3723 - val_loss: 1.7102
Epoch 15/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 63ms/step - accuracy: 0.3909 - loss: 1.5877 - val_accuracy: 0.4069 - val_loss: 1.6711
Epoch 16/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 68ms/step - accuracy: 0.4159 - loss: 1.4758 - val_accuracy: 0.4892 - val_loss: 1.5823
Epoch 17/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 66ms/step - accuracy: 0.4332 - loss: 1.3998 - val_accuracy: 0.4589 - val_loss: 1.7062
Epoch 18/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 65ms/step - accuracy: 0.4625 - loss: 1.3307 - val_accuracy: 0.4589 - val_loss: 1.6307
Epoch 19/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 2s 66ms/step - accuracy: 0.4919 - loss: 1.2586 - val_accuracy: 0.5065 - val_loss: 1.5112
Epoch 20/20
29/29 ━━━━━━━━━━━━━━━━━━━━ 3s 65ms/step - accuracy: 0.5081 - loss: 1.1772 - val_accuracy: 0.4675 - val_loss: 1.5575
```

- **Accuracy and Loss Visualization**



- **Classification Report and Confusion Matrix**

	precision	recall	f1-score	support
angry	0.54	0.58	0.56	38
calm	0.47	0.71	0.56	38
disgust	0.39	0.42	0.41	38
fearful	0.52	0.56	0.54	39
happy	0.33	0.08	0.12	39
neutral	0.00	0.00	0.00	19
sad	0.21	0.11	0.14	38
surprised	0.40	0.79	0.53	39
accuracy			0.43	288
macro avg	0.36	0.41	0.36	288
weighted avg	0.38	0.43	0.38	288



- **Sample output:**

```
Recognized Speech:  Kids are talking by the door.
```

```python
#Perform Text-to-Speech Conversion Using gTTS
from gtts import gTTS
from IPython.display import Audio, display

result = f"The recognized speech is {text}. The predicted emotion is {emotion}."

tts = gTTS(result, lang="en")
tts.save("output.mp3")

display(Audio("output.mp3"))
```

